



Преподаватель:

Коляда

Никита Владимирович

Обратная связь:

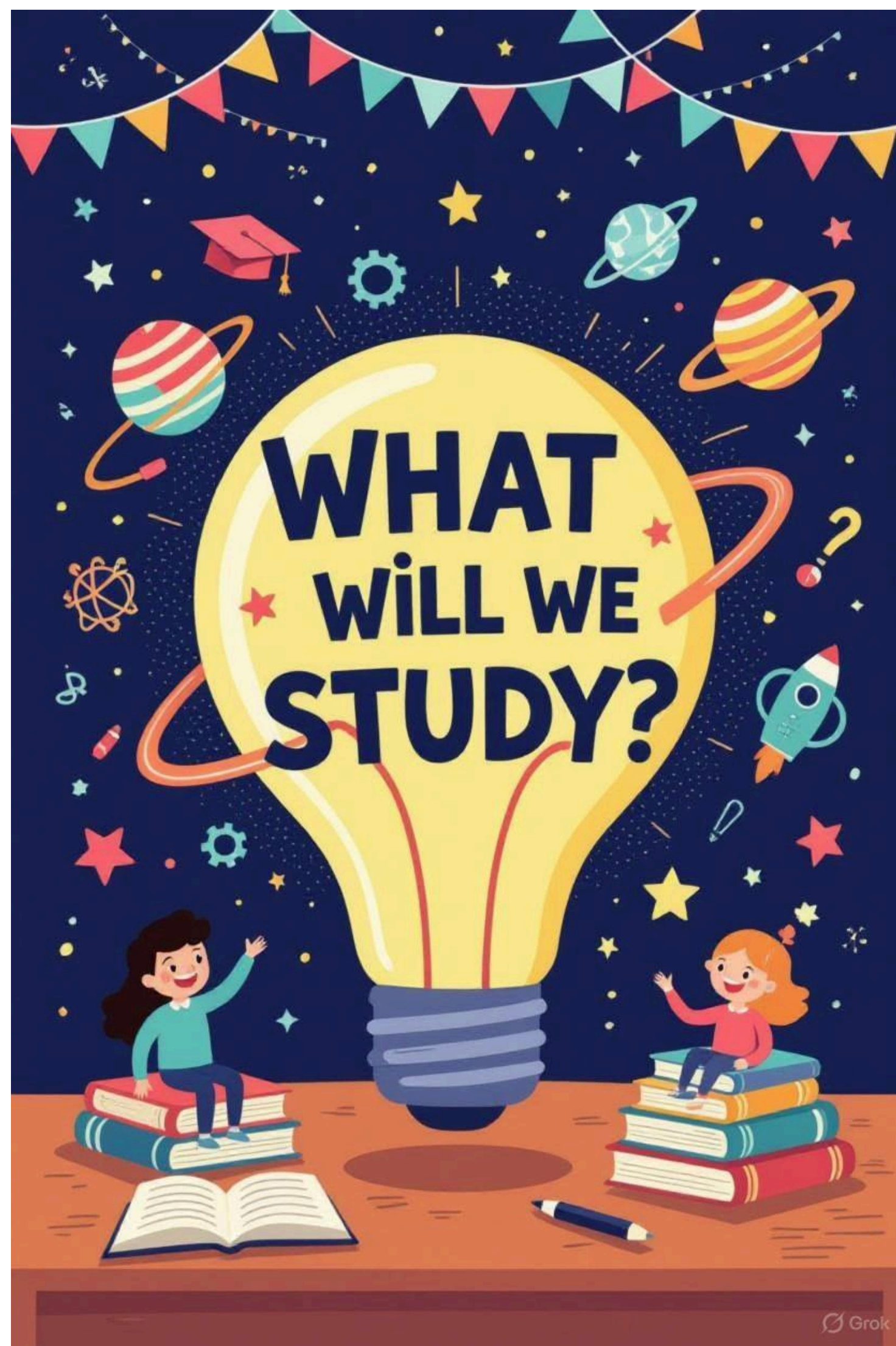
- сообщения на inStudy

ПРОЕКТИРОВАНИЕ ER - ДИАГРАММЫ



ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?

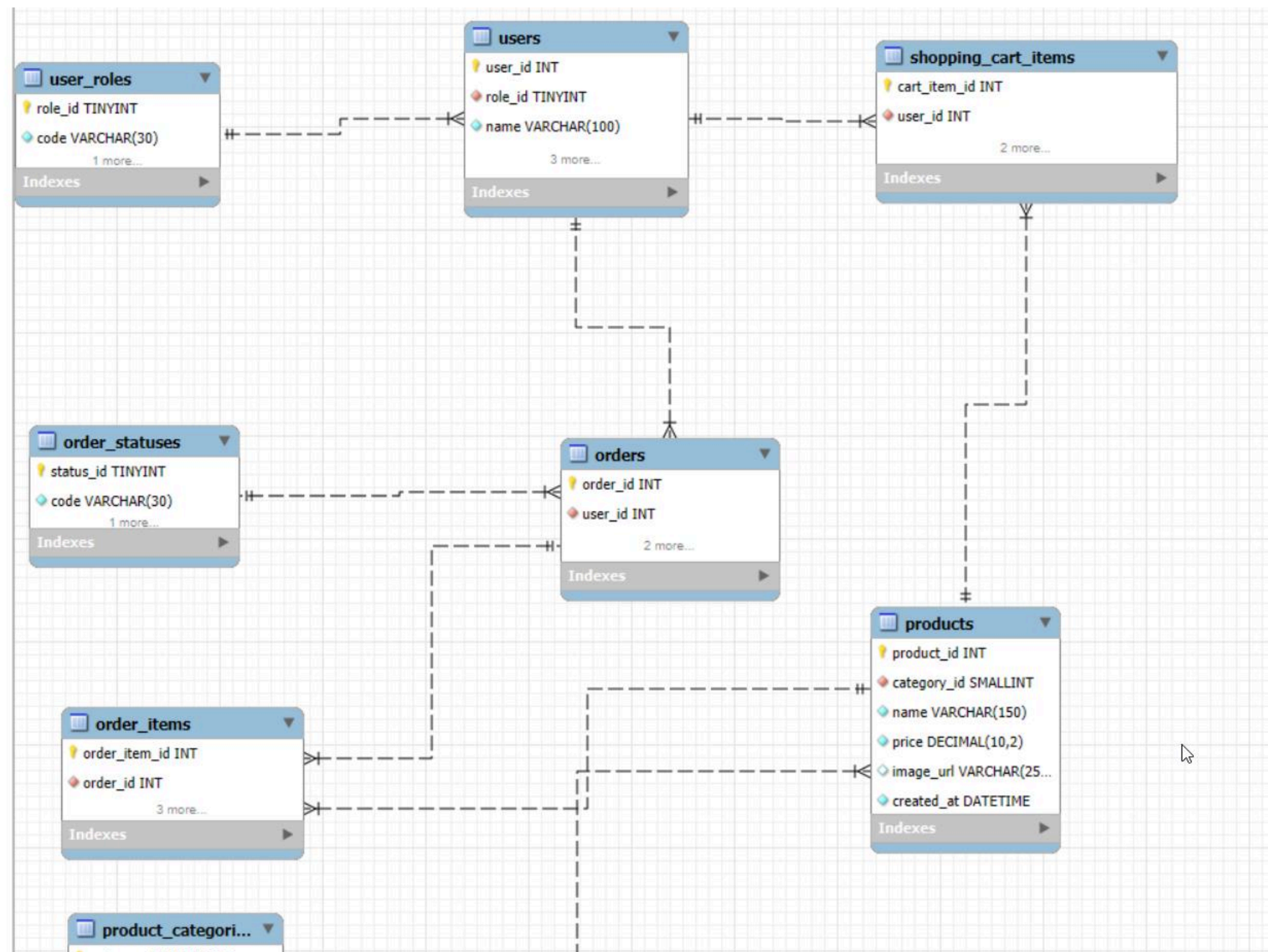


КЛЮЧЕВЫЕ ТЕМЫ

1. Анализ предметной области интернет-магазина
2. Выделение основных сущностей системы
3. Определение справочников (lookup tables)
4. Разделение справочных и транзакционных таблиц
5. Выбор первичных ключей (Primary Key)
6. Использование суррогатных идентификаторов (AUTO_INCREMENT)
7. Подбор типов данных для идентификаторов и атрибутов
8. Добавление уникальных ограничений (UNIQUE)
9. Проектирование связей между сущностями (1:N и M:N)
10. Создание внешних ключей (Foreign Key)
11. Настройка каскадных правил (ON DELETE CASCADE)
12. Реализация связей многие-ко-многим через промежуточные таблицы
13. Добавление бизнес-атрибутов в промежуточные таблицы (quantity, price)
14. Проверка целостности и логики связей в ER-диаграмме

ЦЕЛЬ ПРОЕКТИРОВАНИЯ БАЗЫ ДАННЫХ

ЗАЧЕМ НУЖНА ER-ДИАГРАММА



1. Перед созданием таблиц важно понять, какие объекты существуют в системе и как они связаны.

2. ER-диаграмма — это визуальная модель структуры БД, которая показывает:

- сущности (таблицы),
- их атрибуты (поля),
- связи между ними.

3. Практика:

а. Открыть MySQL Workbench

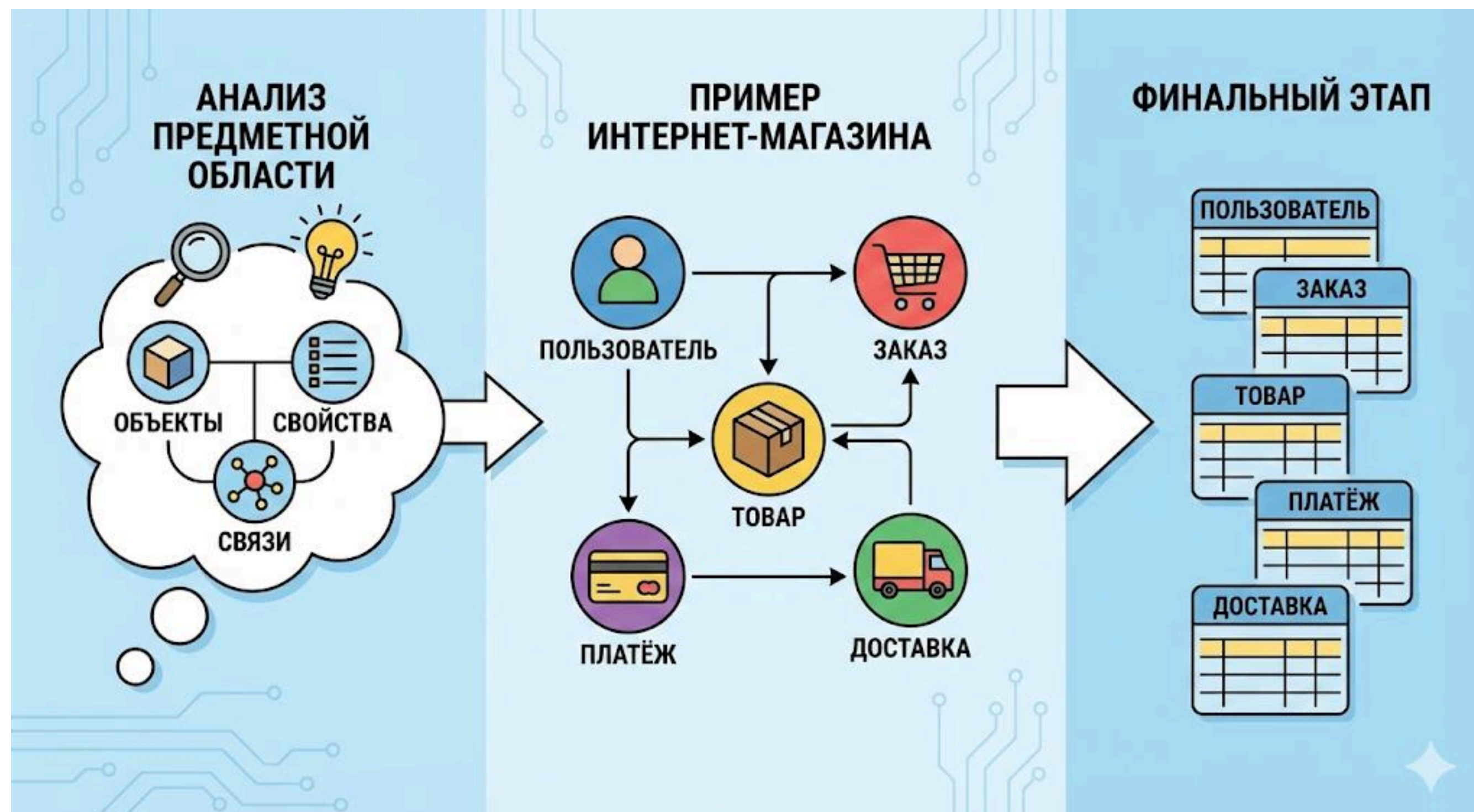
б. Меню: File → New Model

в. В панели слева нажать Add Diagram

г. Откроется пустое поле ER-диаграммы.

АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ (BUSINESS DOMAIN)

НАЧИНАЕМ С АНАЛИЗА ПРЕДМЕТНОЙ ОБЛАСТИ



1. Предметная область – это то, как бизнес работает в реальности.

2. Перед ER-диаграммой нужно ответить на вопросы:

- кто участвует в системе?
- какие действия совершают?
- какие данные нужно хранить?

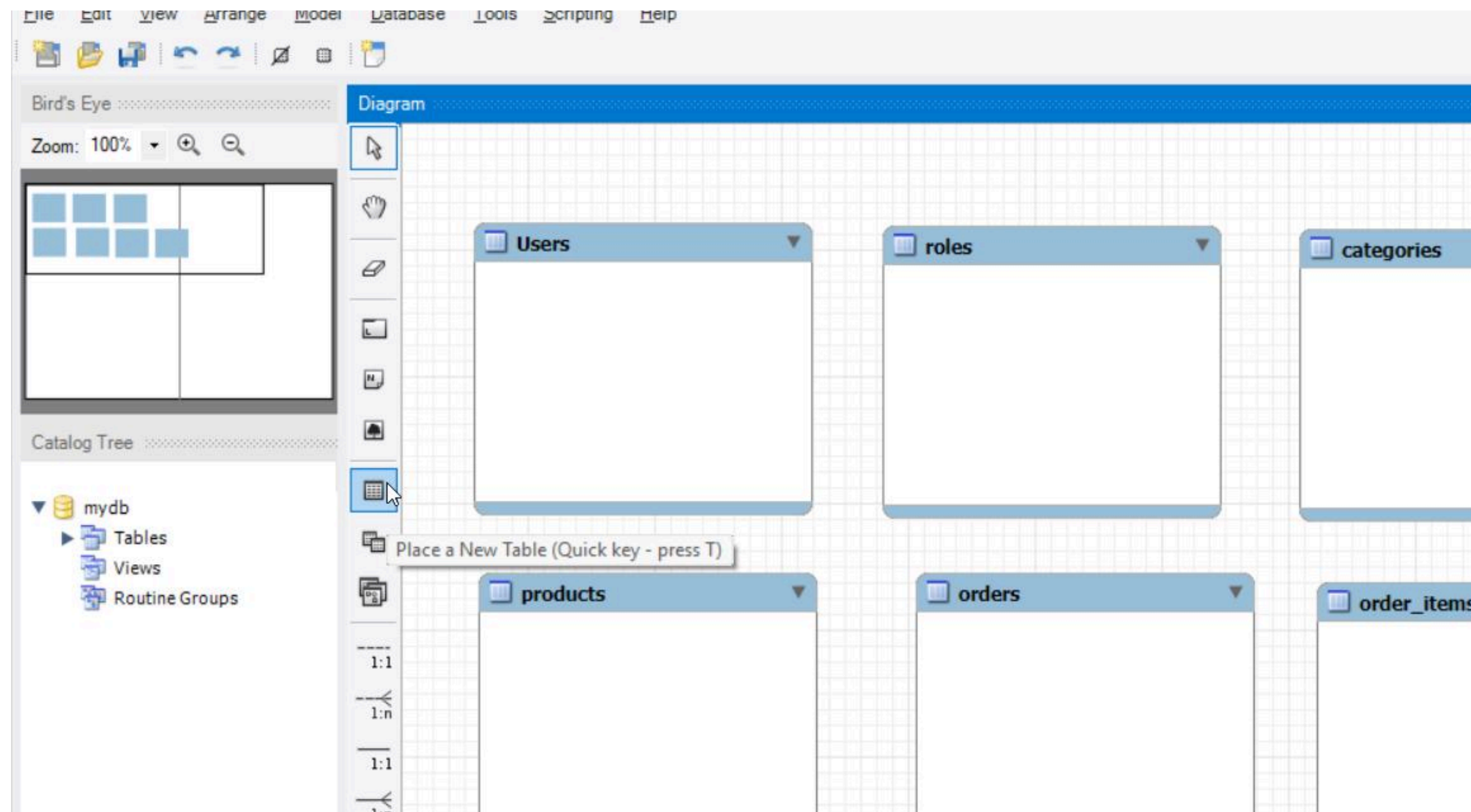
3. Практика

а. Понять, какие объекты должны стать таблицами:

б. пользователь, роль, товар, категория, заказ, позиция заказа, корзина

ВЫДЕЛЕНИЕ СУЩНОСТЕЙ

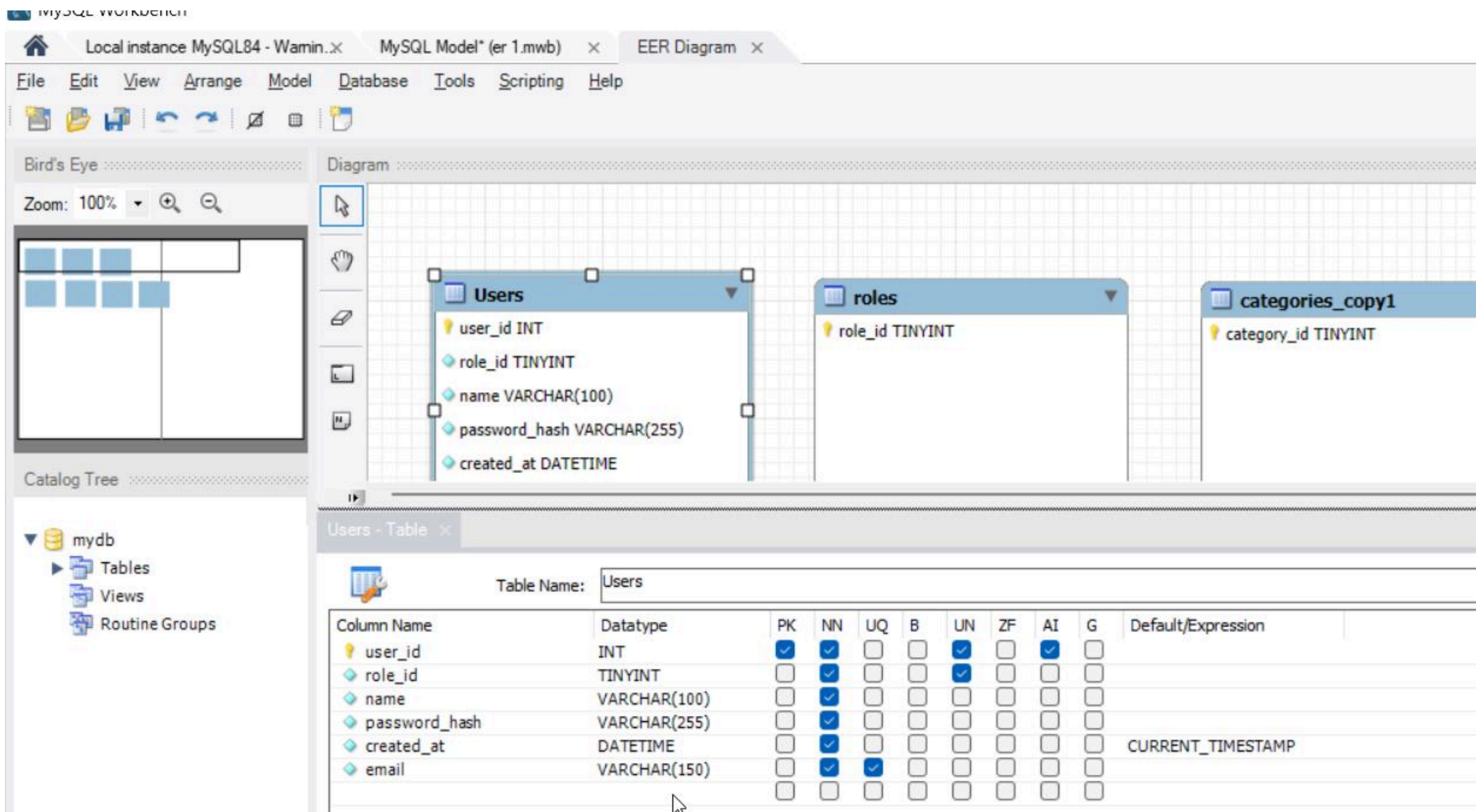
СОЗДАТЬ СУЩНОСТИ НА ДИАГРАММЕ



1. Задача: определить, какие таблицы будут в БД.
2. Создаём таблицы:
3. Справочники(таблицы со стабильным набором значений):
 - user_roles
 - order_statuses
 - product_categories
4. Основные:
 - users
 - products
 - orders
5. Промежуточные:
 - order_items
 - shopping_cart_items
6. Действия:
 - а. Нажать Place a New Table
 - б. Добавить все 8 таблиц (пока пустые)

ТИПЫ ДАННЫХ

ОСНОВНЫЕ ТИПЫ ДАННЫХ ДЛЯ ПОЛЕЙ



Перед созданием таблиц важно понимать типы данных.

Числовые

- TINYINT(0-255) UNSIGNED – маленькие справочники (роли, статусы)
- SMALLINT(0-65 535) UNSIGNED – категории
- INT(0-4+ млрд) UNSIGNED – основные идентификаторы
- DECIMAL(10,2) – цена

Строковые

- VARCHAR(30-255) – имена, email, ссылки, коды
- VARCHAR(255) – хеш пароля, image_url

Дата и время

- DATETIME
- DEFAULT CURRENT_TIMESTAMP

Модификаторы

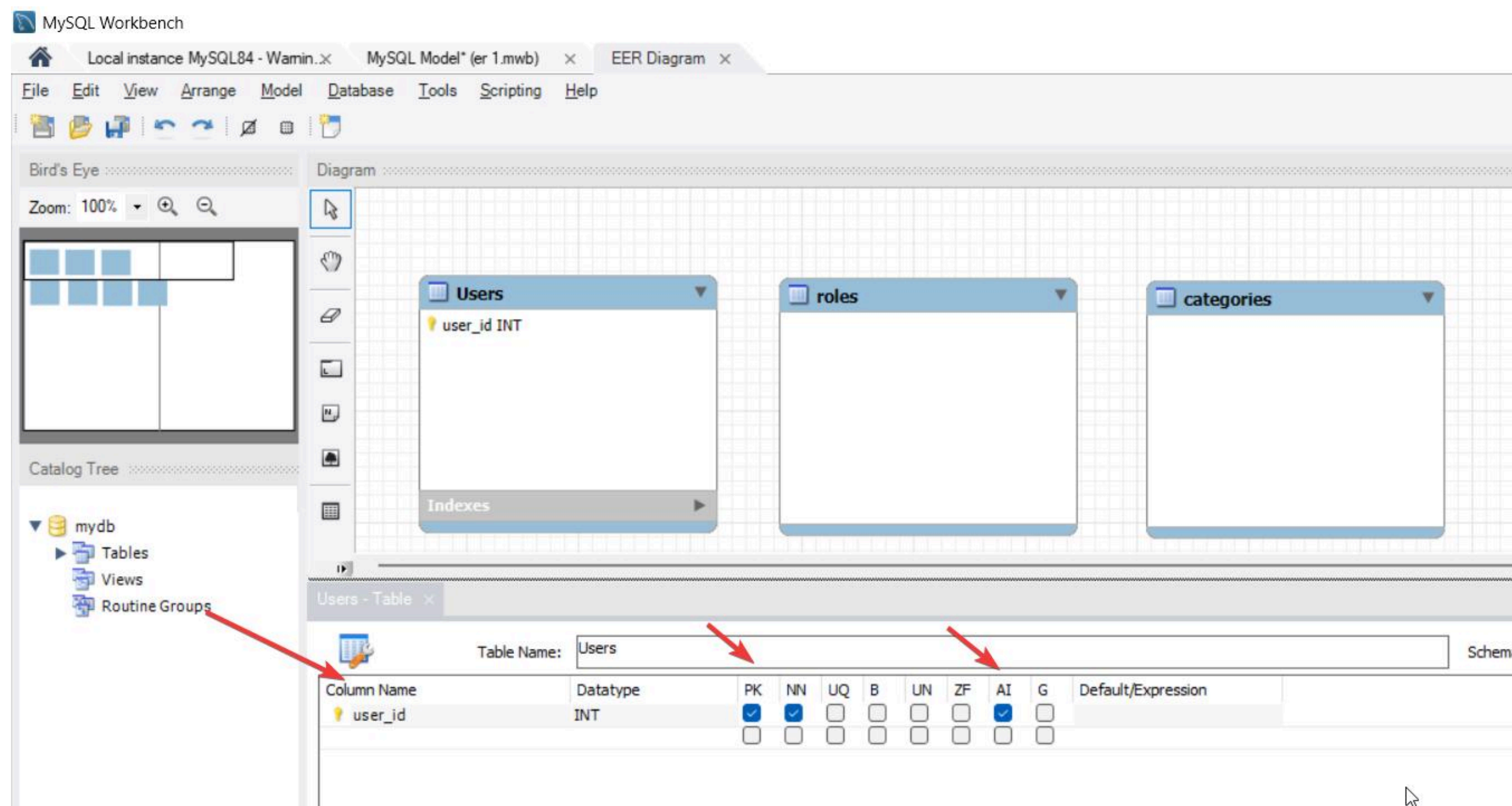
- AUTO_INCREMENT – автоматическое увеличение на 1
- PRIMARY KEY – первичный ключ
- UNIQUE – уникальное значение

Эти типы нужно уметь выбирать под задачу:

- ID → INT + AUTO_INCREMENT
- Цена → DECIMAL, а не FLOAT
- Статусы → маленькие целые числа

ДОБАВЛЕНИЕ ПЕРВИЧНЫХ КЛЮЧЕЙ

ЗАДАТЬ ИДЕНТИФИКАТОРЫ СТРОК



1. Практика (для каждой таблицы)

- Дважды клик по таблице
- Вкладка Columns → Add Column
- Имя: `id_таблицы`
- Тип: `INT` или `TINYINT`
- Отметить:

- PK - PRIMARY KEY
- NN - not null
- AI - AUTO_INCREMENT

2. Примеры (PRIMARY KEY):

- `user_id`
- `product_id`
- `order_id`

ФЛАГИ (АТРИБУТЫ) СТОЛБЦА

ЧТО ОНИ ОЗНАЧАЮТ

Users - Table

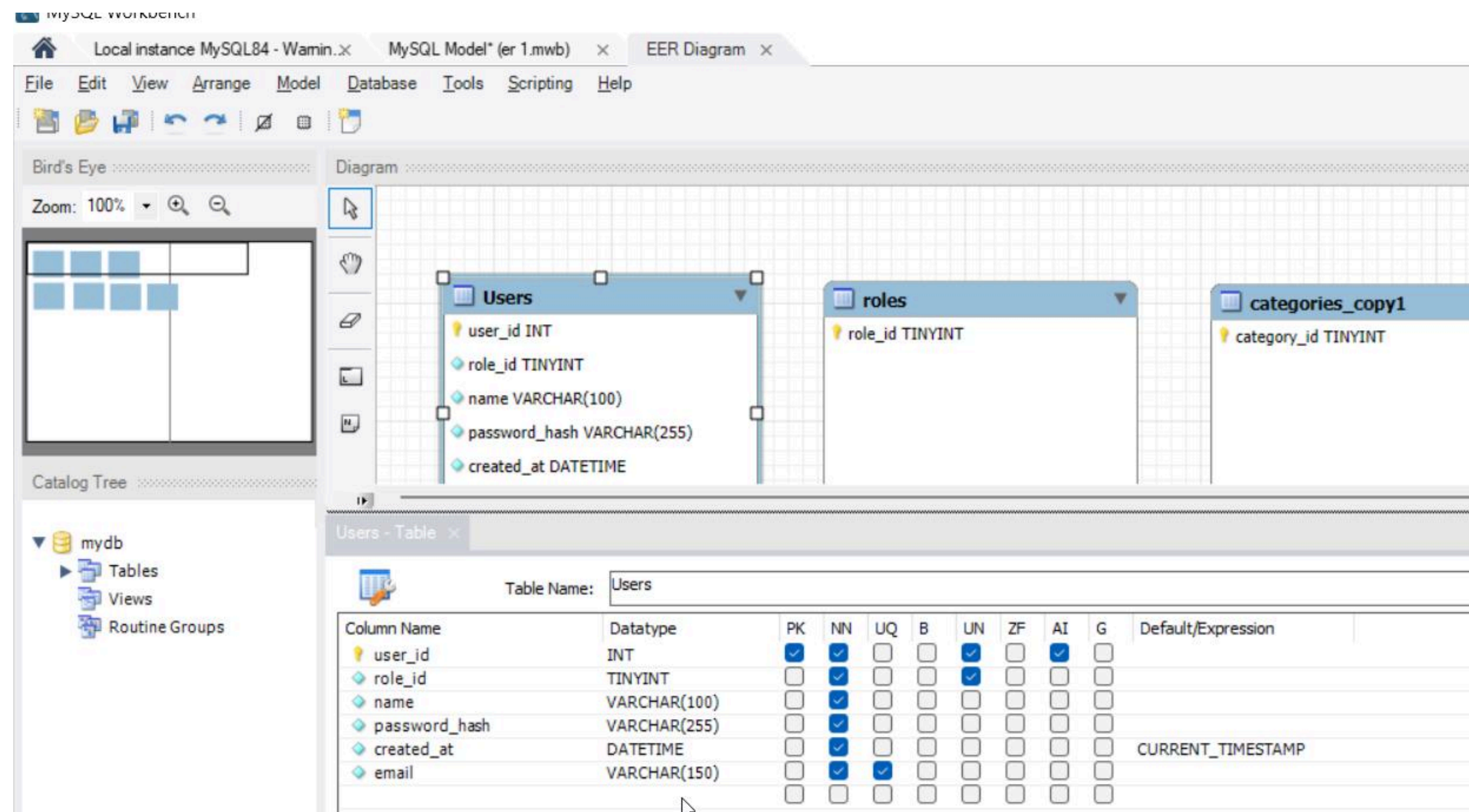
Table Name: Users

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G
user_id	INT	☒	☒	☐	☐	☒	☐	☒	☐
role_id	TINYINT	☐	☒	☐	☐	☒	☐	☐	☐
name	VARCHAR(100)	☐	☒	☐	☐	☐	☐	☐	☐

1. PK — Primary Key
- Первичный ключ. Значение уникально и не может быть NULL.
Используется для идентификации строки в таблице.
2. NN — Not Null
- Запрещает NULL. Поле обязательно должно иметь значение при вставке записи.
3. UQ — Unique
- Уникальное значение в столбце. Повторяющиеся значения запрещены (например, email, код, номер).
4. B — Binary
- Бинарное сравнение для строк. Сравнение выполняется по байтам и с учётом регистра (A ≠ a). Используется редко, например для токенов и хешей.
5. UN — Unsigned
- Беззнаковый тип. Разрешены только неотрицательные значения, увеличивается верхний предел чисел.
6. ZF — Zero Fill
- Дополняет число ведущими нулями при выводе (например, 7 → 0007).
Опция считается устаревшей и почти не применяется.
7. AI — Auto Increment
- Автоматическое увеличение значения. Обычно используется для идентификаторов (id), БД сама подставляет следующее число.
8. G — Generated (Generated Column)
- Вычисляемый столбец. Значение рассчитывается по формуле на основе других колонок и не вводится вручную (например, total = price * quantity).

ДОБАВЛЕНИЕ БИЗНЕС-ПОЛЕЙ

ЗАПОЛНИТЬ ТАБЛИЦЫ АТТРИБУТАМИ



1. Добавляем поля в таблицы:

a. users

- user_id — PK
- role_id — FK
- name
- email (UNIQUE)
- password_hash
- created_at

b. user_roles

- role_id — первичный ключ
- code — уникальный код роли
- title — отображаемое название

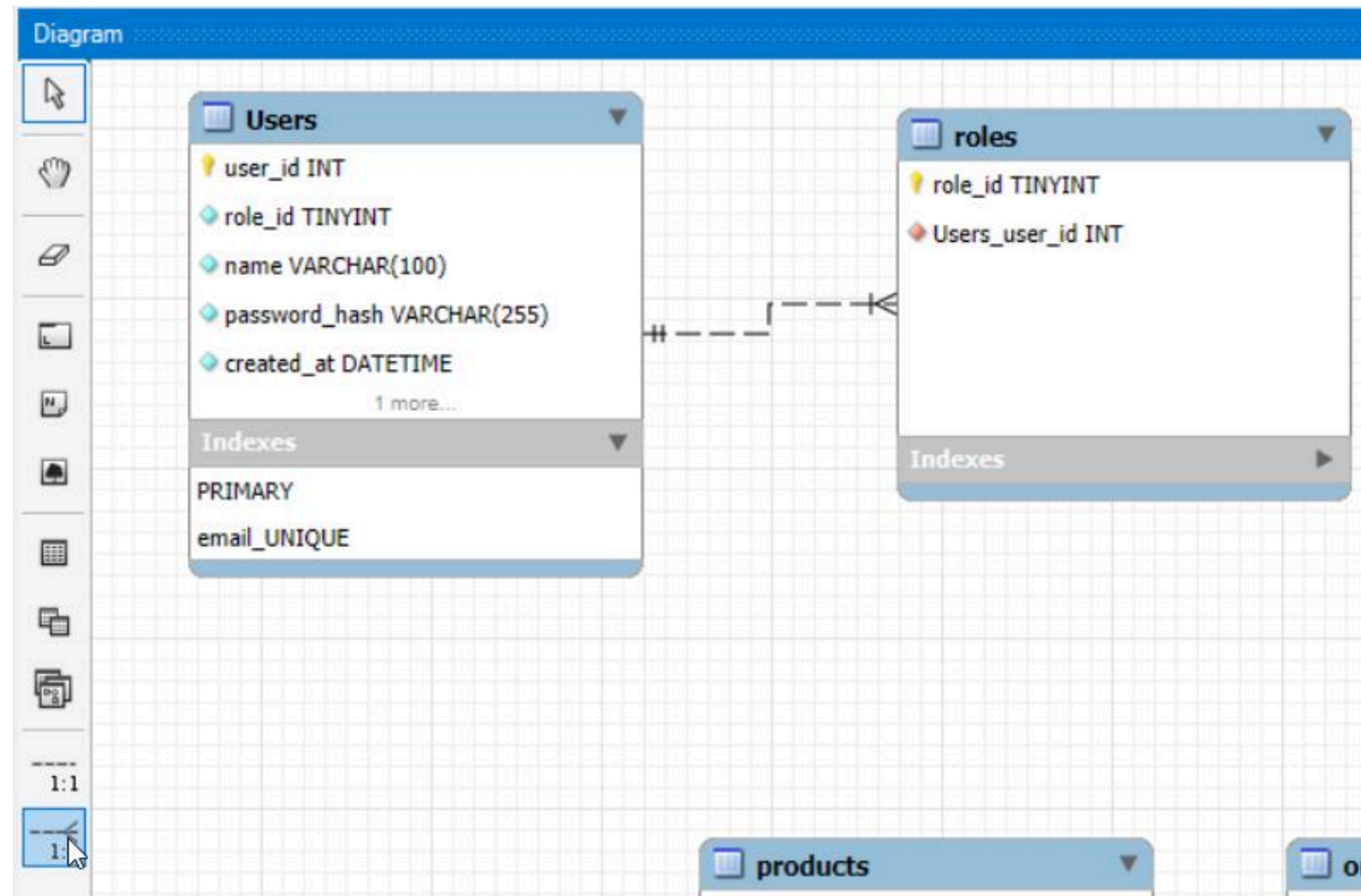
2. Тип связи:

- одна роль → много пользователей (1:N)

3. Остальные таблицы необходимо заполнить самостоятельно

СОЗДАНИЕ СВЯЗЕЙ (FOREIGN KEYS)

НАСТРОИТЬ СВЯЗИ МЕЖДУ ТАБЛИЦАМИ



IDENTIFYING RELATIONSHIP - ИСПОЛЬЗУЕТСЯ,

КОГДА ТАБЛИЦА НЕ ИМЕЕТ СМЫСЛА БЕЗ РОДИТЕЛЬСКИХ СУЩНОСТЕЙ, А ЕЁ РК СТРОИТСЯ НА ИХ КЛЮЧАХ.

NON-IDENTIFYING RELATIONSHIP - ИСПОЛЬЗУЕТСЯ, КОГДА ТАБЛИЦА — ПОЛНОЦЕННАЯ СУЩНОСТЬ СО СВОИМ РК.

1. Практика

- Выбрать инструмент Place a Relationship (1:N) (Identifying or Non-identifying Relationship)
- Клик по родительской таблице
- Клик по дочерней

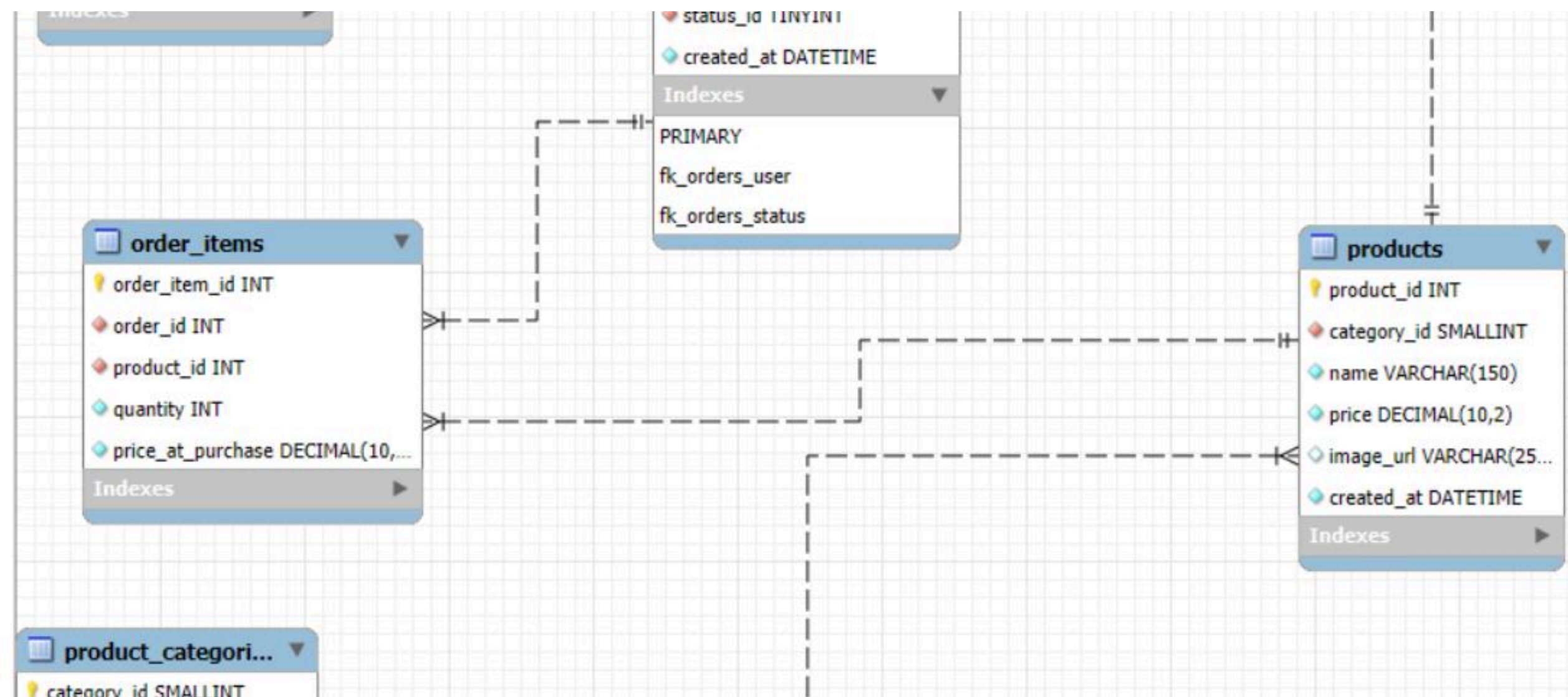
2. Создать связи:

- user_roles → users (Non-Identifying)
- products → order_items (Identifying)

3. Остальные таблицы необходимо заполнить самостоятельно

СВЯЗИ МНОГИЕ-КО-МНОГИМ (М:N)

НАСТРОИТЬ СВЯЗИ



1. В реляционных БД связь М:N нельзя делать напрямую. Нужна промежуточная таблица.
2. Таблица **products**
 - a. `product_id` — PK, `category_id` — FK, `name`, `price`, `image_url`, `created_at`
3. Таблица **order_items** (это таблица-связка):
 - a. `order_item_id` — PK,
 - b. `order_id` — FK,
 - c. `product_id` — FK,
 - d. `quantity`,
 - e. **price_at_purchase** - цена при оплате
4. Почему тут есть цена? Цена может измениться, а заказ должен хранить историю.
 - a. `UNIQUE (order_id, product_id)`
5. Остальные таблицы необходимо заполнить самостоятельно

НАСТРОЙКА UNIQUE И NOT NULL

ОБЕСПЕЧИТЬ КОРРЕКТНОСТЬ ДАННЫХ

t DATETIME

Name: Users

Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Defa
INT	☒	☒	☐	☐	☒	☐	☒	☐	
TINYINT	☐	☒	☐	☐	☒	☐	☐	☐	
VARCHAR(100)	☐	☒	☐	☐	☐	☐	☐	☐	
VARCHAR(255)	☐	☒	☐	☐	☐	☐	☐	☐	
DATETIME	☐	☒	☐	☐	☐	☐	☐	☐	CURI
VARCHAR(150)	☐	☒	☒	☐	☐	☐	☐	☐	
	☐	☐	☐	☐	☐	☐	☐	☐	

1. users.email

a. отметить NN - NOT NULL

b. отметить UQ - UNIQUE

2. product_categories

a. category_name - UNIQUE and NOT NULL

3. order_items

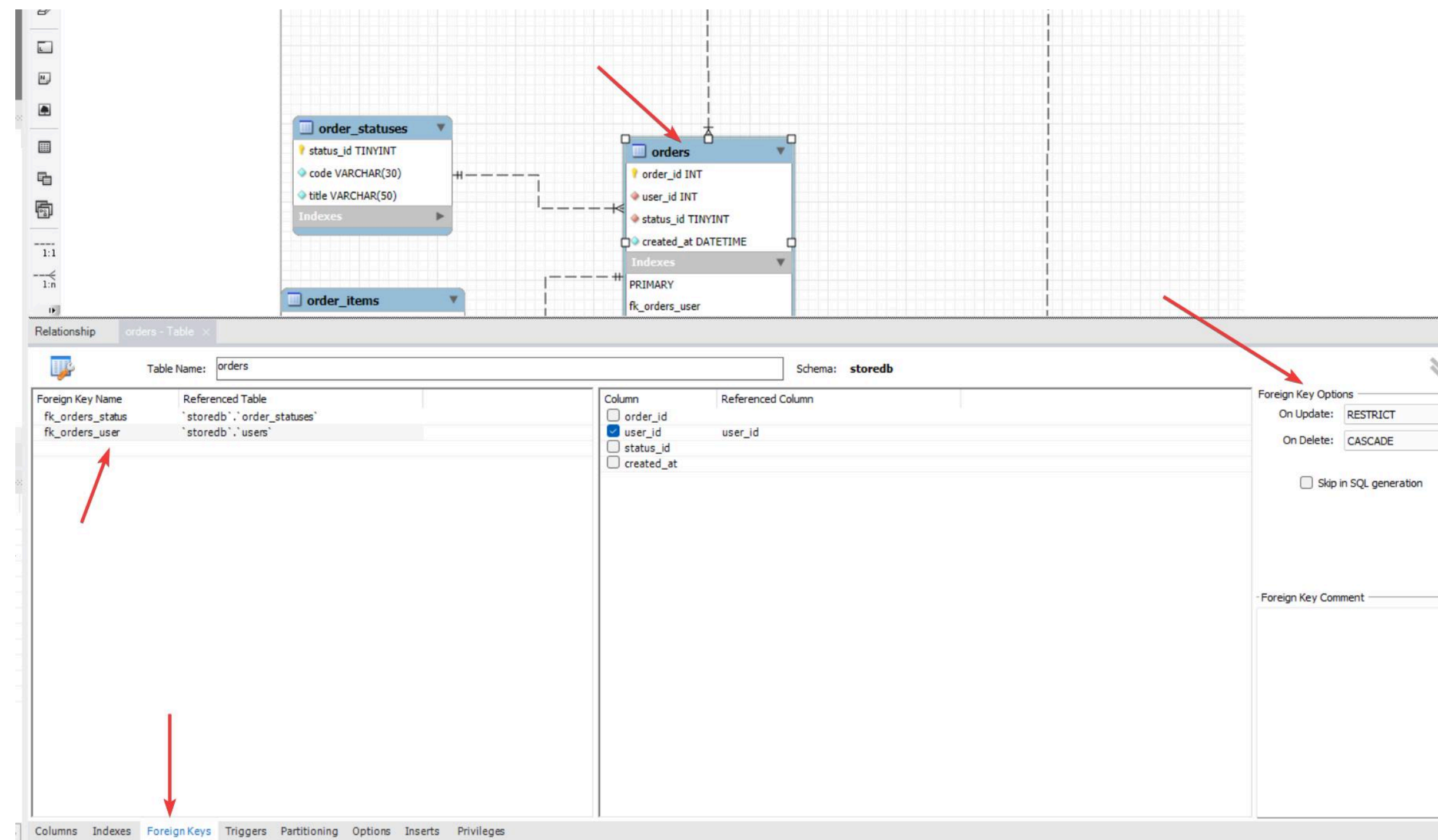
a. NOT NULL - ?

b. UNIQUE (order_id, product_id)

4. Остальные таблицы необходимо заполнить самостоятельно

НАСТРОЙКА КАСКАДНЫХ ПРАВИЛ

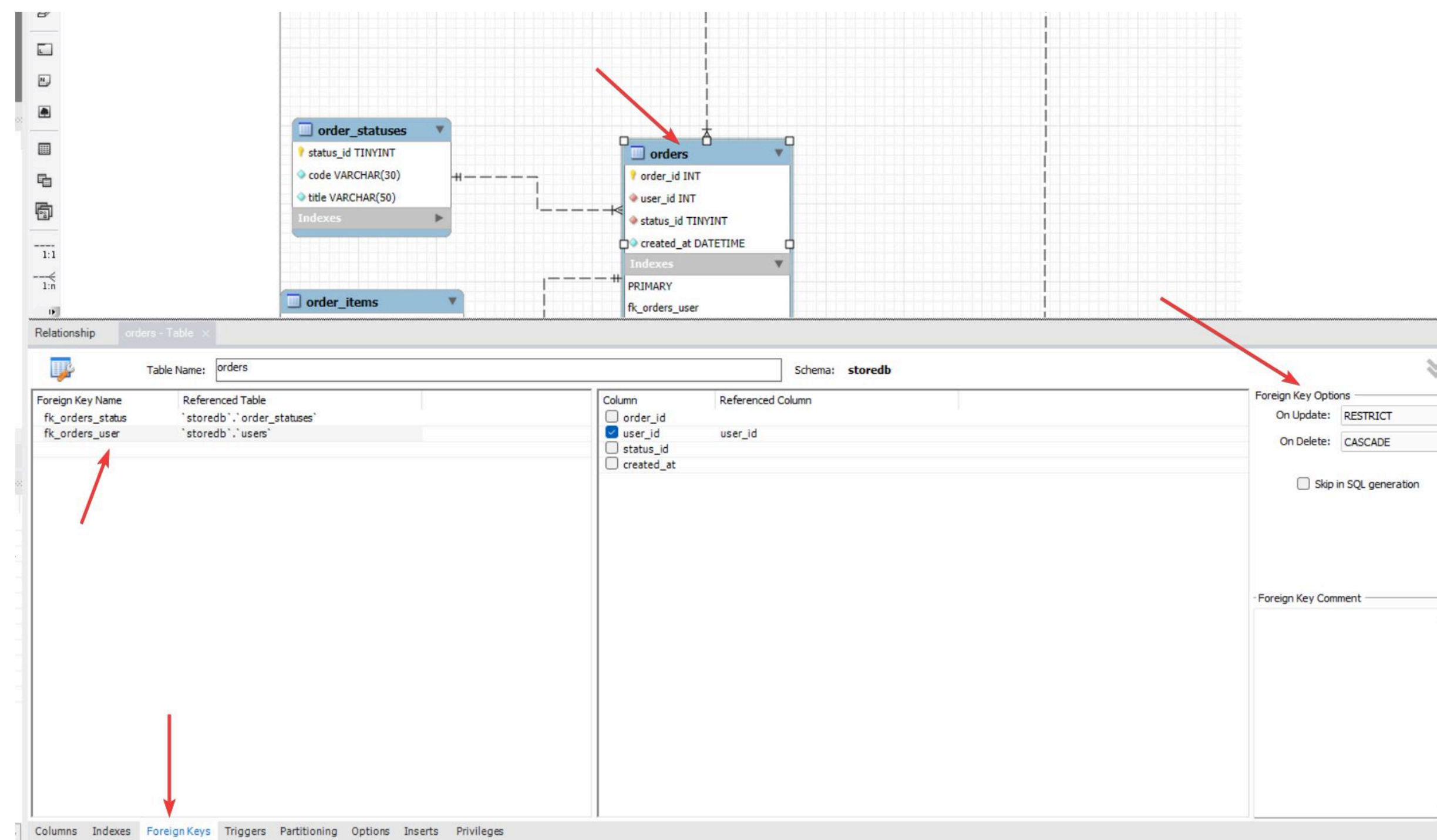
ОПРЕДЕЛИТЬ ПОВЕДЕНИЕ ПРИ УДАЛЕНИИ ДАННЫХ



1. Каскадные правила определяют, что делать с зависимыми строками, если:
 - родительская запись удалена
 - или её ключ изменился
2. Без правил:
 - MySQL запретит удаление
 - или оставит «висячие» ссылки (если FK нет)
3. Не применяются там, где:
 - дочерняя сущность — самостоятельная
 - удаление родителя не должно уничтожать историю
4. **Практика:** Двойной клик по таблице
 - а. Перейди во вкладку "Foreign Keys"
 - б. Выбери нужный внешний ключ
 - с. Настрой правила каскада

НАСТРОЙКА КАСКАДНЫХ ПРАВИЛ

ОПРЕДЕЛИТЬ ПОВЕДЕНИЕ ПРИ УДАЛЕНИИ ДАННЫХ



1. ON DELETE

a. RESTRICT

Запрещает удаление записи, если на неё есть ссылки во внешних ключах.
Сначала нужно удалить или изменить связанные строки в дочерних таблицах.

b. CASCADE

При удалении родительской записи автоматически удаляются все связанные строки в дочерних таблицах.

c. SET NULL

При удалении родительской записи внешний ключ в дочерних строках устанавливается в NULL (только если поле допускает NULL).

d. NO ACTION

Поведение аналогично RESTRICT: удаление не разрешается, если есть связанные записи (в MySQL фактически работает как RESTRICT).

2. ON UPDATE

a. CASCADE

При изменении значения первичного ключа в родительской таблице это изменение автоматически применяется ко всем связанным внешним ключам.

b. RESTRICT

Запрещает изменение ключа, если на него есть ссылки в дочерних таблицах.

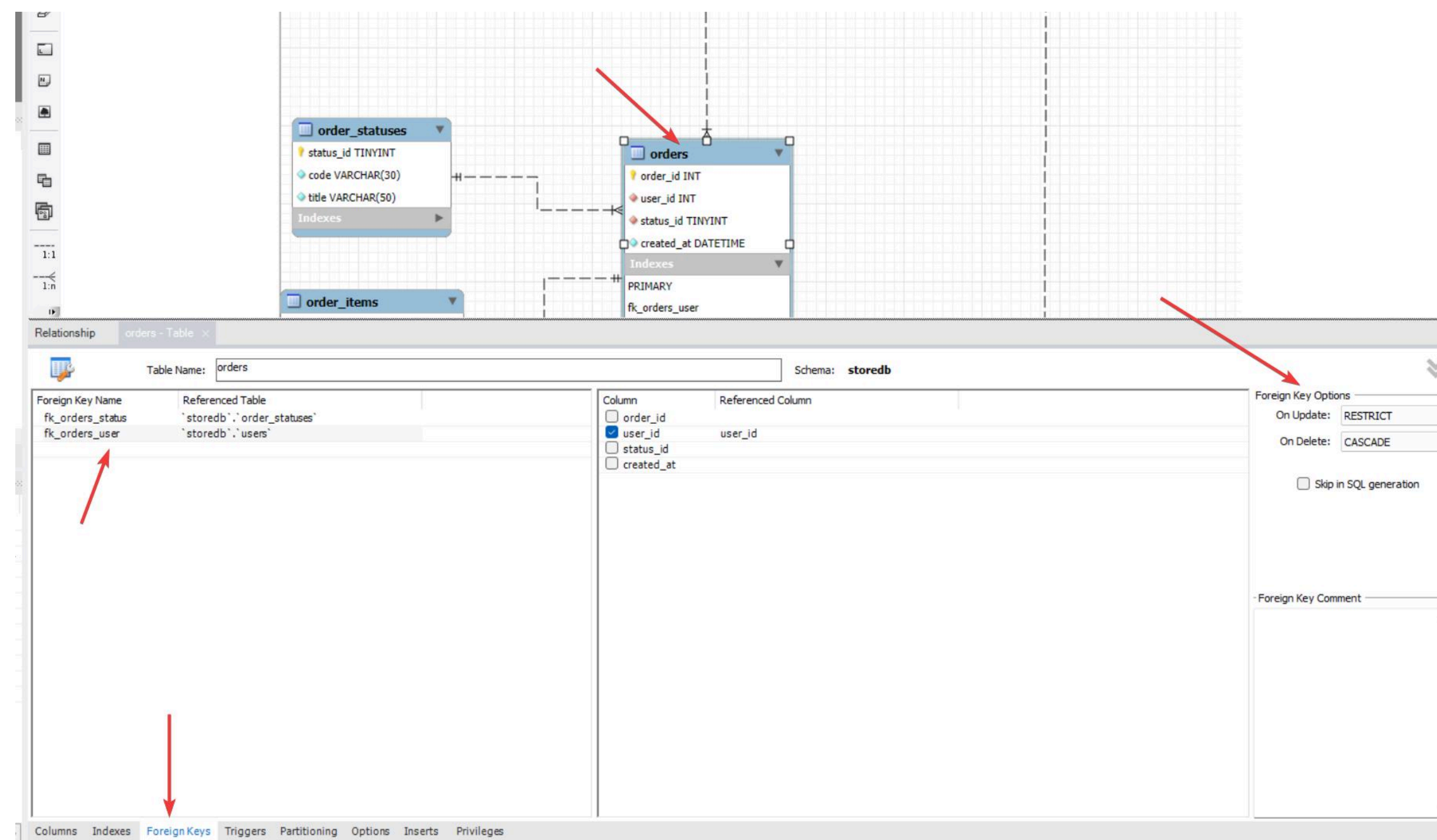
c. NO ACTION

Аналогично RESTRICT: обновление не допускается при наличии связанных записей.

3. На практике: ON UPDATE CASCADE используется редко, потому что первичные ключи обычно не меняются. Чаще настраивают именно правила ON DELETE.

НАСТРОЙКА КАСКАДНЫХ ПРАВИЛ

ОПРЕДЕЛИТЬ ПОВЕДЕНИЕ ПРИ УДАЛЕНИИ ДАННЫХ



1. Примеры каскадных правил

2. **users** → **user_roles**

a. ON DELETE: RESTRICT

b. ON UPDATE: RESTRICT

c. Нельзя удалить роль, если есть пользователи с этой ролью.

Логично: роли — справочник.

3. **order_items** → **orders**

a. ON DELETE: CASCADE

b. ON UPDATE: RESTRICT

c. Удалили заказ → удалились все позиции заказа.

d. без заказа позиции не имеют смысла.

4. **shopping_cart_items** → **users**

a. ON DELETE: CASCADE

b. ON UPDATE: RESTRICT

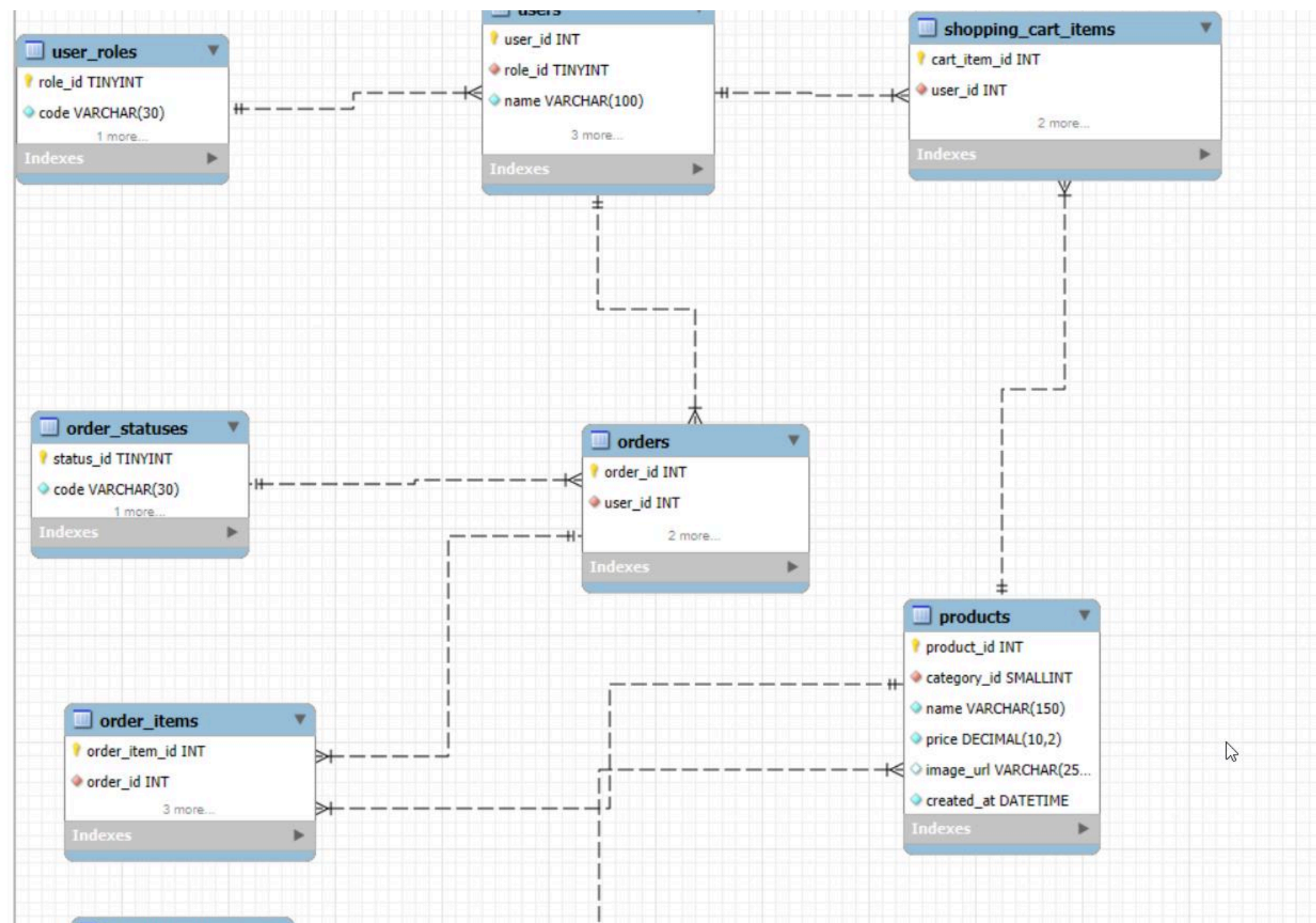
c. Удалили пользователя → корзина очищается.

d. Корзина не имеет смысла без пользователя.

5. Остальные таблицы необходимо заполнить самостоятельно

ИТОГОВАЯ ПРОВЕРКА

ПРОВЕРКА ER-ДИАГРАММЫ НА НАРУШЕНИЯ ЛОГИКИ И ЦЕЛОСТНОСТИ



1. Первичные ключи (PK)

- Каждая таблица должна иметь PK
- Проверить:
 - Уникальность ключа
 - Тип данных соответствует назначению (INT UNSIGNED, TINYINT, и т.д.)
- Отметить отсутствие или несоответствие PK

2. Внешние ключи (FK)

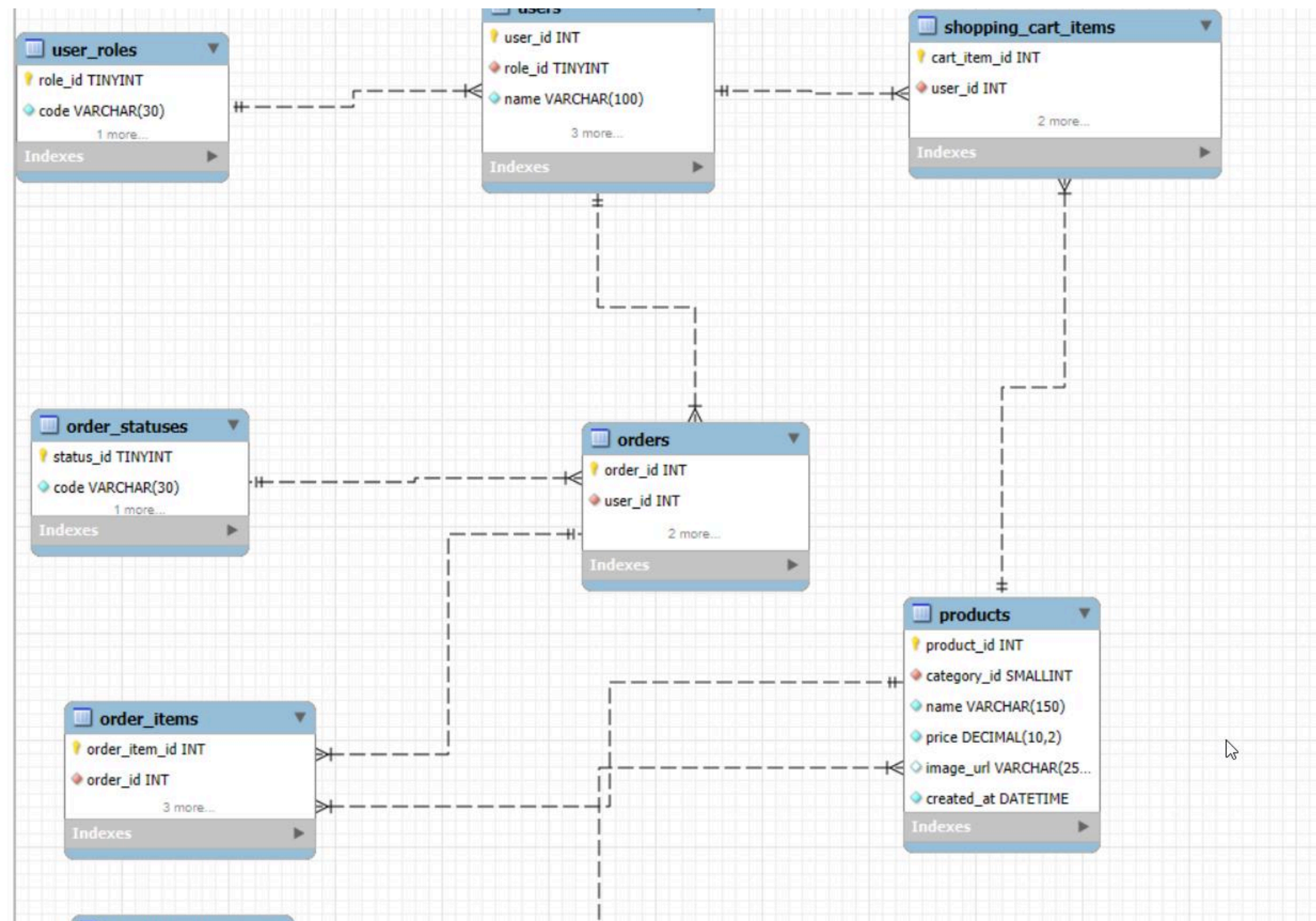
- FK должен ссылаться на существующий PK
- Проверить:
 - Таблица и поле корректны
 - Тип данных FK совпадает с PK
 - Назначены каскадные правила (ON DELETE, ON UPDATE)
- В Workbench: выбрать связь → Properties → Foreign Key → Review

3. Связи 1:N и M:N

- 1:N – FK в дочерней таблице
- M:N – через промежуточную таблицу
- Проверить:
 - Для M:N таблица-связка имеет PK + FK на обе стороны
 - Присутствуют уникальные ключи (UNIQUE) для предотвращения дублирования

ИТОГОВАЯ ПРОВЕРКА

ПРОВЕРКА ER-ДИАГРАММЫ НА НАРУШЕНИЯ ЛОГИКИ И ЦЕЛОСТНОСТИ



4. Справочники

- Таблицы с фиксированным набором значений (user_roles, order_statuses, product_categories)
- Проверить:
 - Используются ли всеми нужными сущностями
 - PK/UNIQUE корректны
 - Нет лишних зависимостей от данных

5. Тип данных

- Строковые поля (VARCHAR) → достаточная длина
- Числовые поля (INT, DECIMAL) → подходят для ID и цен
- Даты (DATETIME) → правильно используются для истории и по умолчанию

6. Каскадные правила

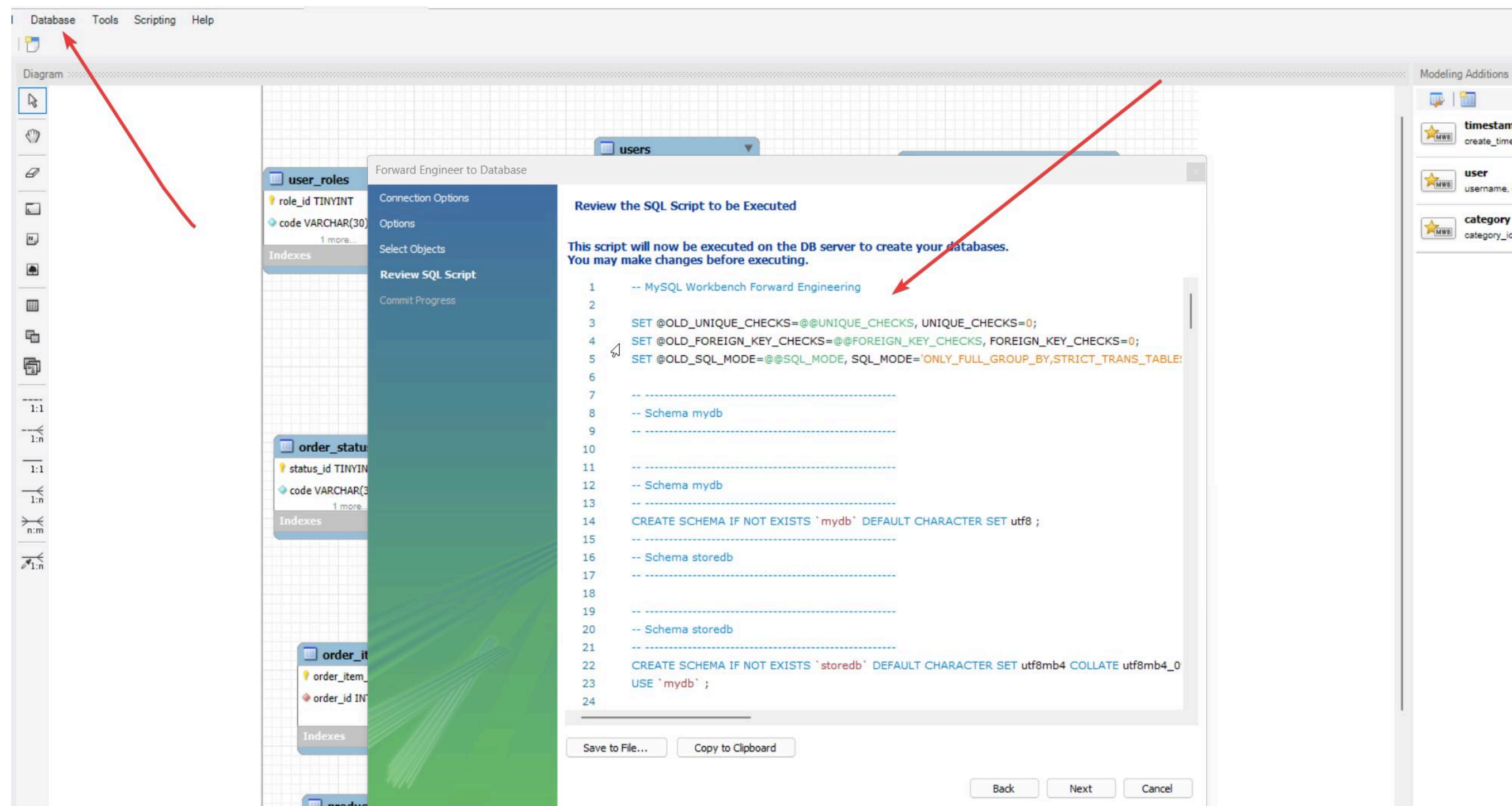
- CASCADE — только там, где дочерняя запись без родителя теряет смысл
- RESTRICT / NO ACTION — для справочников и истории
- SET NULL — если поле допускает NULL и логика позволяет

7. Целостность данных

- Нет «висячих» записей: FK → существующий PK
- Нет дублирующихся значений уникальных полей (UNIQUE)
- Проверка через Workbench: Database → Forward Engineer → Generate SQL → Errors & Warnings

КАК СОЗДАТЬ SQL-СКРИПТ

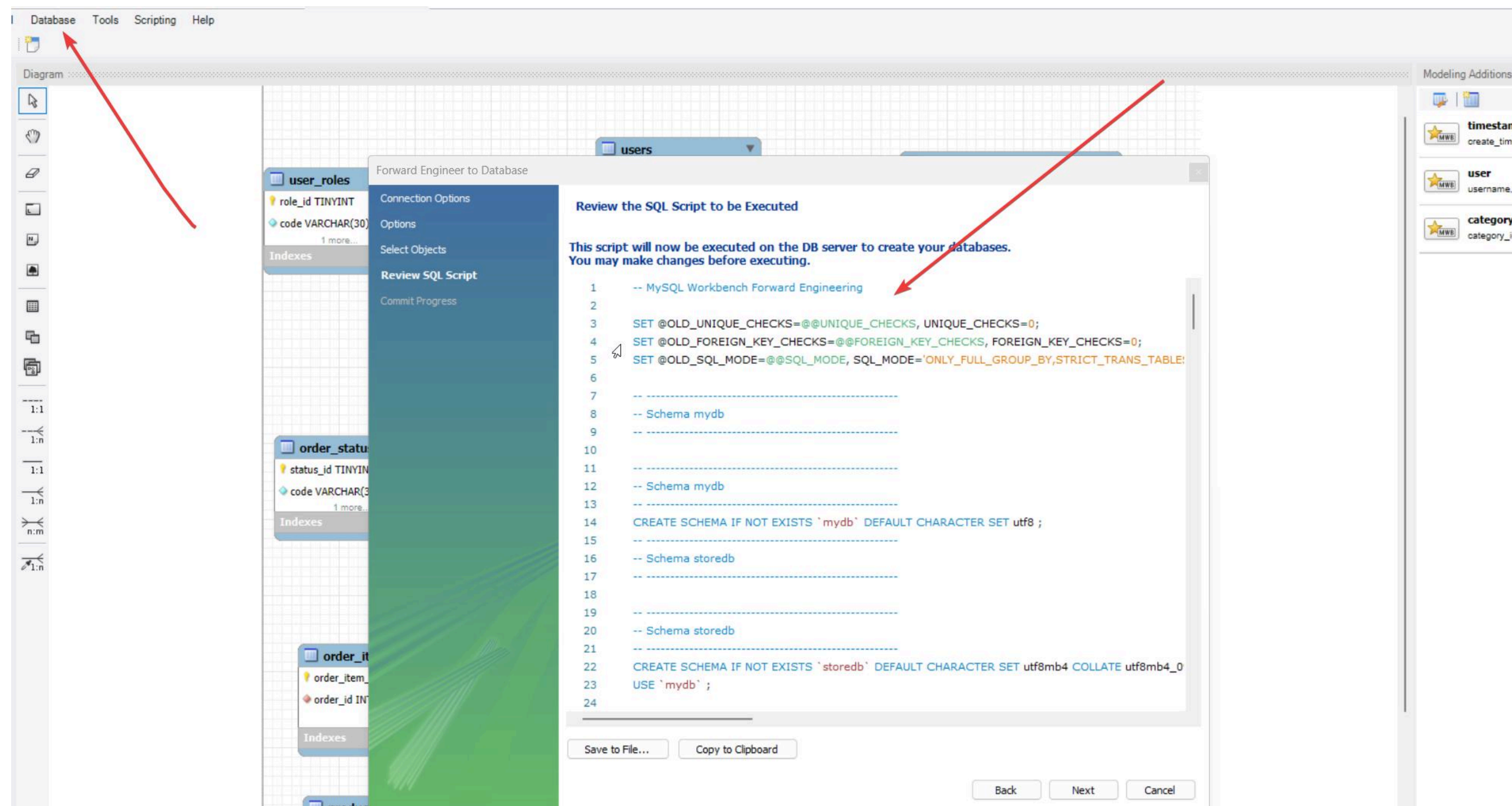
СОЗДАНИЕ SQL-СКРИПТА ИЗ ER-ДИАГРАММЫ



1. Откройте MySQL Workbench и вашу ER-диаграмму.
2. Перейдите в меню: Database → Forward Engineer.
3. Настройте параметры:
 - Выберите нужную схему базы данных
 - Включите создание DROP TABLE, FOREIGN KEY и AUTO_INCREMENT
 - Можно включить создание INSERT для справочников (опционально)
4. Нажмите Next → Generate SQL Script.
5. Сохраните скрипт на диск (storedb.sql).

КАК СОЗДАТЬ SQL-СКРИПТ

СТРУКТУРА ПОЛУЧЕННОГО SQL-СКРИПТА



Структура полученного SQL-скрипта

- Начало скрипта: создание схемы (CREATE SCHEMA) и настройка SQL_MODE
- Справочники (lookup tables)
- Основные сущности
- Промежуточные таблицы (M:N)
- Установки каскадных правил (ON DELETE, ON UPDATE)
- Опциональные вставки данных (INSERT INTO ...)
- В конце: восстановление режимов и проверок

КАК ЗАПУСТИТЬ SQL-СКРИПТ

СОЗДАНИЕ БАЗЫ ДАННЫХ НА СЕРВЕРЕ ИЗ СКРИПТА

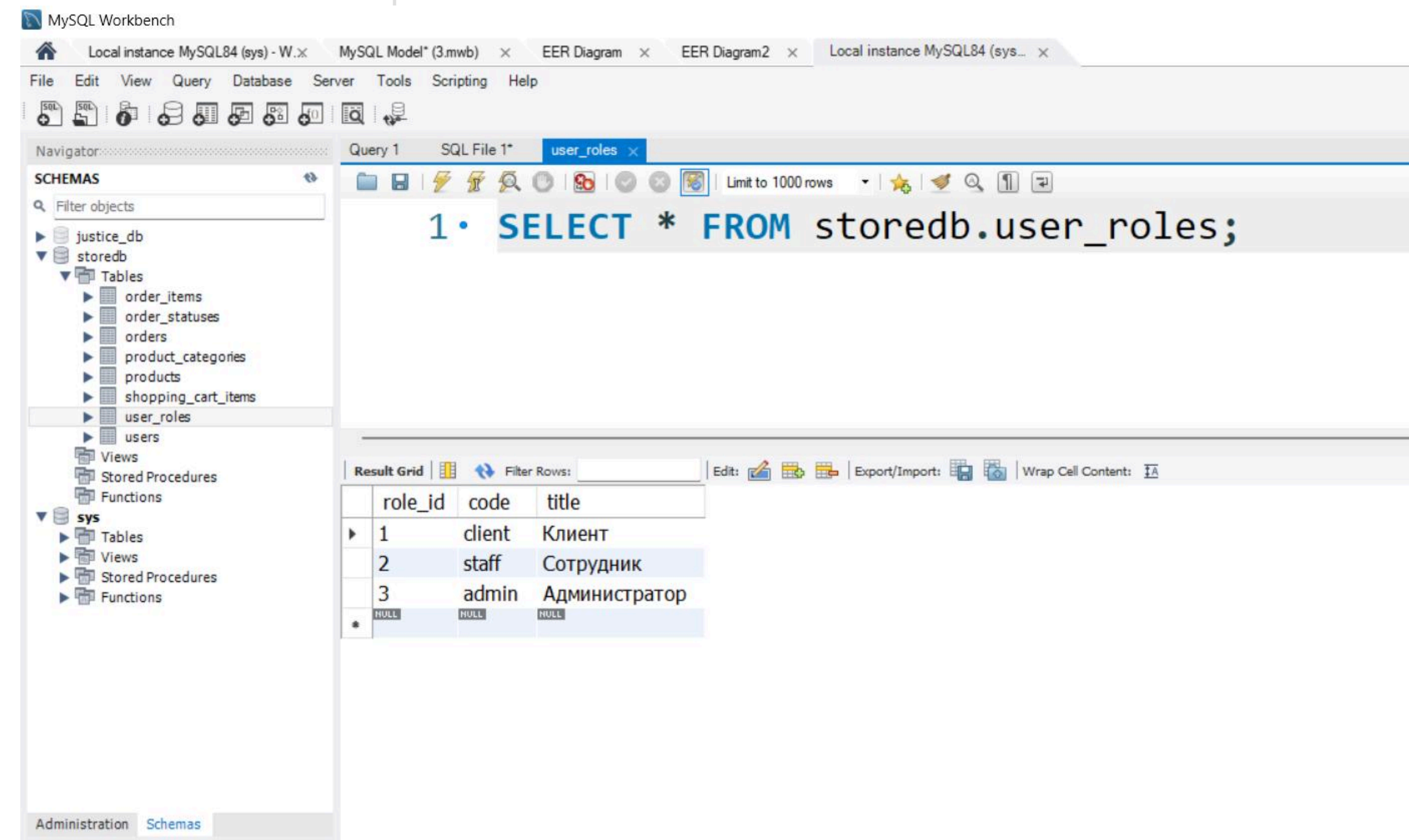
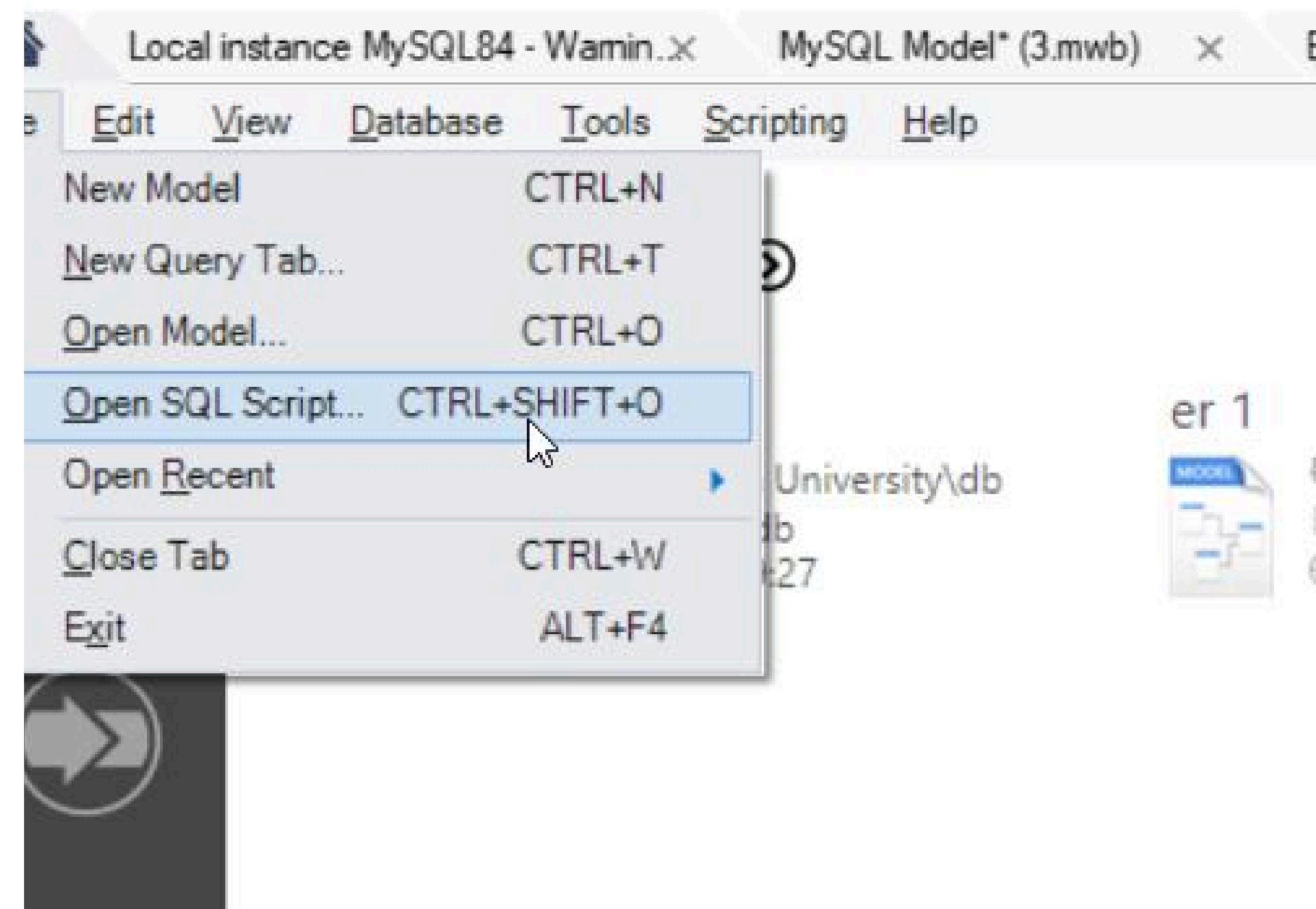
В MySQL Workbench

1. File → Open SQL Script → выберите скрипт

2. Нажмите Execute (Ctrl+Shift+Enter)

- Таблицы создаются в базе данных
- В Workbench можно увидеть их сразу в Schemas → storedb → Tables

MySQL Workbench





ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?